

Neural Networks for Sequences

Chapter 5 · From fixed context to learned memory

Dr. A. Belcaid

Table of contents

1 The memory problem left by n-grams **3**
1.1 A dependency that does not fit the window . . . 3
1.1.1 Worked solution . . . 4
1.2 Why increasing the order is not learned memory . . . 5
1.3 Two limitations that should not be confused . . . 5
1.3.1 Failure 1: sparse evidence . . . 6
1.3.2 Failure 2: discarded evidence . . . 6
1.4 Section synthesis . . . 7

2 Fixed-window neural language models **7**
2.1 Bengio's distributed-representation argument . . . 7
2.2 Building the model from inspectable objects . . . 8
2.2.1 1. Look up one vector per context word . . . 8
2.2.2 2. Concatenate the window . . . 9
2.2.3 3. Compute a hidden representation . . . 9
2.2.4 4. Produce a vocabulary distribution . . . 9
2.3 What the neural window fixes—and what it does not . . . 9

3 Recurrent neural networks **10**
3.1 One recurrent cell . . . 10
3.2 A complete scalar forward pass . . . 11

4 Learning through time **12**
4.1 Backpropagation through one cell . . . 12
4.2 One concrete token loss . . . 13
4.3 Multiple gradient paths add . . . 13
4.4 Backpropagation through time . . . 14

5 Generating structured text with an RNN **15**

6 Why gradients vanish or explode **16**
6.1 The repeated Jacobian product . . . 16
6.2 A scalar calculation . . . 16
6.3 Gradient clipping: a practical guardrail . . . 17

7 Gated recurrent cells **18**
7.1 LSTM: retain, write, expose . . . 18
7.1.1 Retain or forget . . . 18
7.1.2 Prepare a write . . . 18
7.1.3 Update memory additively . . . 19
7.1.4 Expose part of memory . . . 19
7.2 GRU: reset, propose, interpolate . . . 19
7.2.1 Reset the old state before proposing a candidate . . . 19
7.2.2 Interpolate between old and proposed state . . . 19

8 Neural machine translation **20**
8.1 Encoder–decoder recurrence . . . 20

9 Conditional language models **21**
9.1 Training seq2seq end to end . . . 22

10 The fixed-context bottleneck **22**
10.1 Chapter synthesis . . . 23
10.2 References . . . 23

A fixed-window language model can use several recent words, but it cannot decide to remember an important word for an unknown amount of time. This chapter develops the machinery needed to replace that rigid window with a learned state. We will move from neural language models to recurrent computation, backpropagation through time, gated memory, and finally sequence-to-sequence learning.

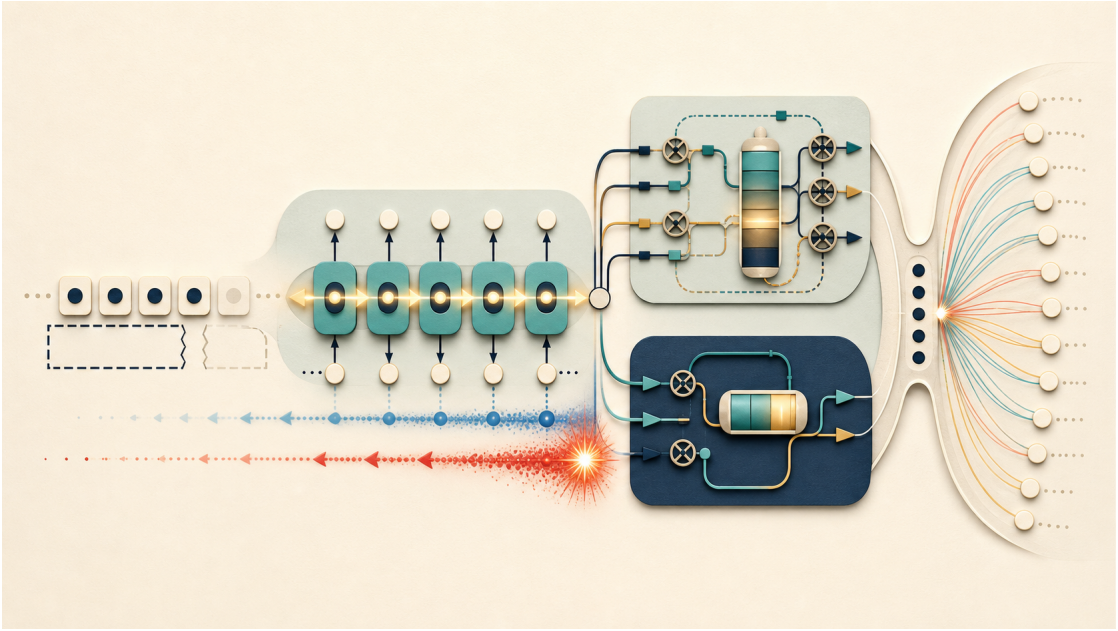


Figure 1: The chapter journey from a fixed context window through recurrent state, temporal credit assignment, gated memory, and the fixed-context bottleneck that motivates attention.



Figure 2: Each part repairs a limitation exposed by the preceding part.

Learning objectives

After this chapter, you should be able to:

1. distinguish the sparsity problem from the fixed-memory problem of n-gram models;
2. construct a fixed-window neural language model from embeddings and shared parameters;
3. compute the forward equations of an RNN and interpret its hidden state;
4. explain how backpropagation through time accumulates parameter gradients;
5. diagnose vanishing and exploding gradients and apply gradient clipping;
6. compare the information paths in LSTM and GRU cells;
7. explain encoder–decoder and conditional language-model training; and
8. identify the fixed-context bottleneck that motivates attention.

1 The memory problem left by n-grams

The previous chapter constructed language models from counts. An n-gram model estimates a next-token distribution from a fixed suffix of the available history:

$$P(w_t \mid w_{1:t-1}) \approx P(w_t \mid w_{t-n+1:t-1}). \tag{1}$$

This approximation is computationally useful, but it imposes a strict memory policy: **retain the most recent $n-1$ tokens and discard everything earlier**. The policy depends only on position. It cannot retain a distant word because that word is important.

1.1 A dependency that does not fit the window

Keep the following sentence visible throughout the example:

> The **keys** to the cabinet ___ on the table.

The exact task is to predict the missing verb. The plural noun *keys* determines that the correct continuation is *are*. The singular noun *cabinet* is closer to the blank, but it is not the subject.

The token sequence available before the prediction is:

Table 1: The important grammatical feature occurs at position 2, while the nearest noun occurs at position 5.

Position	1	2	3	4	5
Token	the	**keys**	to	the	cabinet
Role	determiner	plural subject	connector	determiner	intervening noun

A bigram model retains only the final token:

$$P(w_6 \mid w_{1:5}) \approx P(w_6 \mid \text{cabinet}).$$

The model therefore asks, “Which word often follows *cabinet*?” It cannot ask, “Which verb agrees with the subject of this clause?” The plural feature was present in the sentence, but it was discarded before prediction.

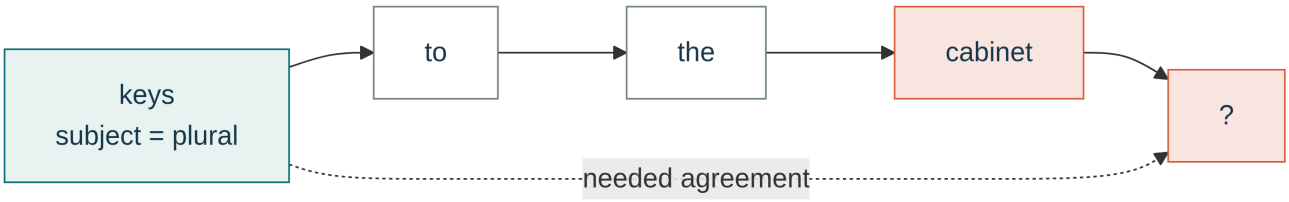


Figure 3: The sentence contains the needed feature, but the bigram representation exposes only the final token to the predictor.

> **! Try it before reading the solution**
>
> Suppose the model is a trigram. Its retained history contains two tokens.
>
> 1. What exact history does it see before predicting the verb?
> 2. Does that history contain the number of the subject?
> 3. What minimum n-gram order would include *keys* in this particular sentence?

1.1.1 Worked solution

The two-token trigram history is:

```
the cabinet
```

It still does not contain *keys*, so it does not expose the subject’s plural number. The distance from *keys* to the missing verb is four token transitions:

```
keys → to → the → cabinet → ___
```

To include *keys* when predicting the blank, the retained history must contain four tokens. That requires a **5-gram** model:

$$P(w_6 \mid w_{1:5}) \approx P(w_6 \mid w_2, w_3, w_4, w_5).$$

For this one sentence, increasing the order repairs the missing-input problem. It does not provide a general solution: another sentence can insert five, ten, or fifty tokens between the subject and its verb.

1.2 Why increasing the order is not learned memory

Increasing n changes the length of the window, not the rule used to select information. The model still retains a fixed suffix and discards the rest.

If the vocabulary contains $|V|$ tokens, an order-n model may need to distinguish as many as $|V|^{n-1}$ histories and $|V|^n$ complete n-grams. For $|V| = 50{,}000$,

$$(50{,}000)^5 = 3.125 \times 10^{23}$$

possible 5-grams exist.

The number is not a claim that a system will store a dense table of that size. It exposes the combinatorial space in which observations become sparse: longer histories are more informative when repeated, but exact repetitions become less likely (Jurafsky and Martin 2026, ch. 3).

Table 2: A longer window can capture this dependency but cannot adapt to dependencies of unknown length.

Order	Retained history before the blank	Contains *keys*?	General solution?
Bigram	`cabinet`	No	No
Trigram	`the cabinet`	No	No
5-gram	`keys to the cabinet`	Yes	No
Larger fixed n	a longer suffix	Sometimes	No

1.3 Two limitations that should not be confused

N-gram models exhibit two related but distinct failures.

1.3.1 Failure 1: sparse evidence

Suppose the training corpus contains:

> the cat rested

but never contains:

> the dog rested

A count model may assign the second phrase zero probability even though *cat* and *dog* occur in similar linguistic environments. This is a **generalization problem**: the model treats different word identities as unrelated table entries.

Smoothing redistributes probability mass and prevents unseen events from receiving probability zero. However, ordinary smoothing still does not give *cat* and *dog* a learned geometric relationship.

1.3.2 Failure 2: discarded evidence

In the agreement sentence, the relevant word *keys* occurred, but it fell outside the active window. This is a **memory problem**: information needed at prediction time is absent from the model input.

Smoothing cannot reconstruct a feature that the representation has already removed. The difference can be summarized as follows:

Table 3: Generalization and memory require different repairs.

Question	Sparse evidence	Discarded evidence
Was the useful information in the active representation?	Yes, but the event was rare or unseen	No
Typical symptom	zero or unreliable count	prediction based on the wrong local cue
Can smoothing help?	Yes	No
Needed architectural change	share evidence across similar words	carry selected information across time

> ⚠ **Common misconception**
>
> "Use a larger n" is not a mechanism for long-term memory. It is a decision to preserve a longer **fixed** suffix, with a rapidly growing and increasingly sparse context space.

1.4 Section synthesis

An n-gram model forgets by position: once a token leaves the window, its information is unavailable. A useful sequence model should instead learn a compact representation that:

- shares statistical evidence across related words;
- is updated as each new token arrives;
- preserves information when it remains useful; and
- discards information when it no longer matters.

The first neural step will address the generalization problem by replacing symbolic word identities with learned vectors and shared parameters. It will also make the remaining memory limitation easier to see.

> **! Question leading to the next section**
>
> Can a neural language model learn that *cat* and *dog* are similar while keeping the exact inputs, vectors, and scores inspectable?

2 Fixed-window neural language models

The first repair is to stop treating every word as an unrelated symbol. A neural language model learns a vector for each word and reuses a shared prediction network across every context. Words that support similar predictions can acquire similar vectors, allowing evidence to be shared across related contexts (Bengio et al. 2003).

2.1 Bengio's distributed-representation argument

Consider two contexts:

```
the cat rested
the dog rested
```

A count table stores separate entries for *cat* and *dog*. A neural model instead looks up learned embeddings

$$\mathbf{e}_{\text{cat}} = C\mathbf{1}_{\text{cat}}, \qquad \mathbf{e}_{\text{dog}} = C\mathbf{1}_{\text{dog}},$$

where $C \in \mathbb{R}^{d_e \times |V|}$ is an embedding matrix and each one-hot vector selects one column. If training makes the two embeddings nearby, then a shared downstream network reacts similarly to both words.

What is being shared?

The model does not copy a probability from one sentence to another. It learns reusable features. A hidden unit that responds to “animate subject” or “plural noun” can fire for many words instead of requiring a separate count for every surface form.

The original neural probabilistic language model jointly learned word vectors and the probability function that used them. Its architecture is reproduced in Figure 4 for historical reference.

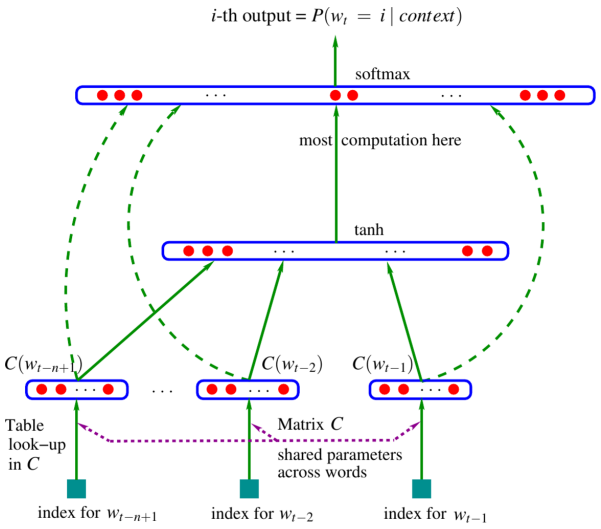


Figure 4: The neural probabilistic language-model architecture. Reproduced from Bengio et al. (2003), Fig. 1.

2.2 Building the model from inspectable objects

Use a four-word context to predict the next word:

```
the students opened their  →  ?
```

Let every embedding have dimension d_e. The model constructs four visible objects.

2.2.1 1. Look up one vector per context word

$$\mathbf{e}^{(1)} = C\mathbf{1}_{\text{the}}, \quad \mathbf{e}^{(2)} = C\mathbf{1}_{\text{students}}, \quad \mathbf{e}^{(3)} = C\mathbf{1}_{\text{opened}}, \quad \mathbf{e}^{(4)} = C\mathbf{1}_{\text{their}}.$$

The same matrix C performs every lookup. Word identity chooses a column; position does not create a new embedding table.

2.2.2 2. Concatenate the window

$$\mathbf{x} = \begin{bmatrix} \mathbf{e}^{(1)} \\ \mathbf{e}^{(2)} \\ \mathbf{e}^{(3)} \\ \mathbf{e}^{(4)} \end{bmatrix} \in \mathbb{R}^{4d_e}. \tag{2}$$

Concatenation preserves word order because each position occupies a different block of $mathbfx$.

2.2.3 3. Compute a hidden representation

$$\mathbf{a} = W\mathbf{x} + \mathbf{b}_h, \qquad \mathbf{h} = \tanh(\mathbf{a}), \tag{3}$$

with $W \in \mathbb{R}^{d_h \times 4d_e}$ and $mathbfb_h \in \mathbb{R}^{d_h}$.

2.2.4 4. Produce a vocabulary distribution

$$\mathbf{z} = U\mathbf{h} + \mathbf{b}_o, \qquad \hat{\mathbf{y}} = \text{softmax}(\mathbf{z}), \tag{4}$$

where $U \in \mathbb{R}^{|V| \times d_h}$. The output contains one probability per vocabulary item. Plausible continuations might include *books* and *laptops*; implausible continuations remain present with low probability.

Table 4: The dimensions make the fixed-window assumption visible in the parameters.

Object	Shape	Interpretation
C	$d_e \times \lvert V \rvert$	shared word-embedding table
$mathbfx$	$4d_e$	ordered four-word context
W	$d_h \times 4d_e$	context-to-hidden transformation
U	$\lvert V \rvert \times d_h$	hidden-to-vocabulary transformation
$\hat{\mathbf{y}}$	$\lvert V \rvert$	next-word distribution

2.3 What the neural window fixes—and what it does not

The neural model improves generalization in two ways:

1. embeddings allow related words to share evidence; and
2. shared matrices replace a table containing one independent parameter set per observed n-gram.

The memory policy, however, remains fixed. If the window contains k embeddings,

$$\mathbf{x} \in \mathbb{R}^{kd_e}, \qquad W \in \mathbb{R}^{d_h \times kd_e}.$$

Adding one more position adds $d_h d_e$ parameters to W. More importantly, concatenation assigns a distinct block of W to every position. The model does not apply one position-independent rule repeatedly through time.

> ⚠ **The boundary moved; it did not disappear**
>
> A four-word neural window forgets the fifth-most-recent word just as completely as a 5-gram model. Distributed representations repair exact-match generalization, not unbounded memory.

The required architecture should accept an arbitrary number of inputs while keeping its parameter shapes fixed. That requirement leads to recurrence.

3 Recurrent neural networks

A recurrent neural network (RNN) maintains a **hidden state** that is updated after every input. Instead of concatenating a separate block for every position, it applies the same transformation at each time step.

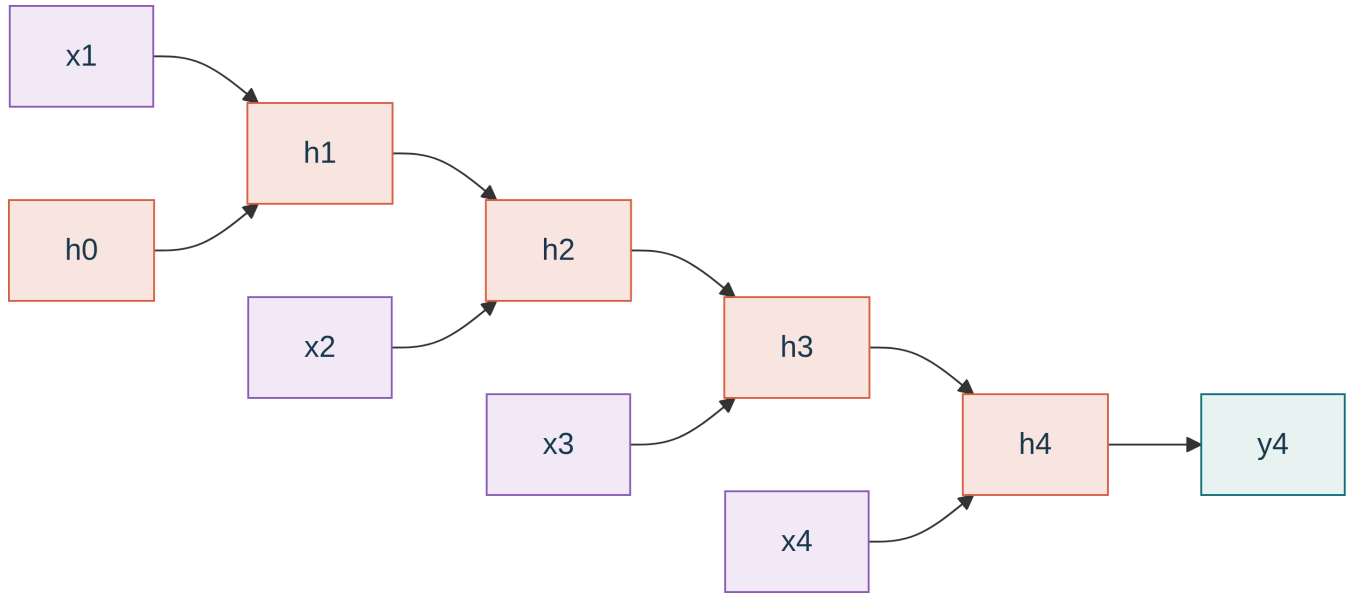


Figure 5: Unfolding reveals repeated applications of one state-update rule. The graph grows with the sequence, but the parameter set does not.

3.1 One recurrent cell

For input $mathbf{x}_t \in \mathbb{R}^{d_x}$ and previous state $mathbf{h}_{t-1} \in \mathbb{R}^{d_h}$, a simple tanh RNN computes

$$\begin{aligned}\mathbf{a}_t &= W_x\mathbf{x}_t + W_h\mathbf{h}_{t-1} + \mathbf{b}_h,\\ \mathbf{h}_t &= \tanh(\mathbf{a}_t),\\ \mathbf{z}_t &= U\mathbf{h}_t + \mathbf{b}_o,\\ \hat{\mathbf{y}}_t &= \mathrm{softmax}(\mathbf{z}_t).\end{aligned} \tag{5}$$

The matrices have fixed shapes:

$$W_x \in \mathbb{R}^{d_h \times d_x}, \quad W_h \in \mathbb{R}^{d_h \times d_h}, \quad U \in \mathbb{R}^{|V| \times d_h}.$$

The subscript t appears on activations, not parameters. Every time step reuses W_x, W_h, U, and the biases.

State is not a stored sentence

$\mathbf{h}_t$ is a learned summary produced by repeated transformations. It is not a list of previous words, and no coordinate is guaranteed to have a human-readable meaning. Training determines which aspects of the history become useful features.

3.2 A complete scalar forward pass

Before working with vectors, inspect a one-dimensional RNN. Let

$$h_t = \tanh(w_x x_t + w_h h_{t-1} + b),$$

with

$$w_x = 0.8, \qquad w_h = 0.5, \qquad b = 0, \qquad h_0 = 0.$$

Suppose two scalar inputs are $x_1 = 1$ and $x_2 = -0.5$.

Try it first

Compute a_1, h_1, a_2, h_2. Keep at least three decimal places.

At the first step,

$$a_1 = 0.8(1) + 0.5(0) = 0.8, \qquad h_1 = \tanh(0.8) \approx 0.664.$$

At the second step, the new input and previous state meet:

$$a_2 = 0.8(-0.5) + 0.5(0.664) = -0.400 + 0.332 = -0.068,$$

$$h_2 = \tanh(-0.068) \approx -0.068.$$

Table 5: The state combines new evidence with a transformed summary of earlier evidence.

Step	x_t	recurrent contribution $w_h h_{t-1}$	input contribution $w_x x_t$	a_t	h_t
1	1.0	0.000	0.800	0.800	0.664
2	−0.5	0.332	−0.400	−0.068	−0.068

The state can continue for five or five hundred inputs without changing the number or shapes of the parameters. The price is sequential computation: h_t cannot be computed until h_{t-1} exists.

4 Learning through time

Training an RNN requires derivatives through the reused recurrent computation. We first derive one local backward step, then connect those steps across the unfolded sequence.

4.1 Backpropagation through one cell

For a one-hot target $\mathbf{y}_t$, the cross-entropy loss is

$$J^{(t)} = -\sum_{v \in V} y_{t,v} \log \hat{y}_{t,v}. \tag{6}$$

Softmax and cross-entropy simplify the derivative with respect to logits:

$$\delta_{z,t} = \frac{\partial J^{(t)}}{\partial \mathbf{z}_t} = \hat{\mathbf{y}}_t - \mathbf{y}_t. \tag{7}$$

The output matrix sends that error into hidden-state space:

$$\mathbf{g}_{h,t}^{\text{local}} = U^\top \delta_{z,t}.$$

For $\mathbf{h}_t = \tanh(\mathbf{a}_t)$,

$$\delta_{a,t} = \mathbf{g}_{h,t} \odot (1 - \mathbf{h}_t \odot \mathbf{h}_t). \tag{8}$$

The same pre-activation error produces every parameter gradient entering the cell:

$$\begin{aligned}
\frac{\partial J}{\partial W_x} &= \delta_{a,t}\mathbf{x}_t^\top, \\
\frac{\partial J}{\partial W_h} &= \delta_{a,t}\mathbf{h}_{t-1}^\top, \\
\frac{\partial J}{\partial \mathbf{b}_h} &= \delta_{a,t}, \\
\frac{\partial J}{\partial \mathbf{h}_{t-1}} &= W_h^\top \delta_{a,t}.
\end{aligned} \tag{9}$$

The final line is what makes temporal credit assignment possible: error at time t becomes an error signal for the preceding state.

4.2 One concrete token loss

After the context

```
the students opened their
```

suppose the observed target is *books* and the model assigns

$$\hat{y}_{t,\text{books}} = 0.62.$$

Because the target is one-hot, all terms in Equation 6 except the target entry vanish:

$$J^{(t)} = -\log(0.62) \approx 0.478.$$

A larger probability on the correct word produces a smaller loss. The loss is attached to this output, but its gradient reaches the state, the current input path, and the preceding state.

4.3 Multiple gradient paths add

An intermediate state $\mathbf{h}_t$ has at least two downstream uses:

1. it produces the current output and loss $J^{(t)}$; and
2. it influences $\mathbf{h}_{t+1}$ and therefore all later losses.

Backpropagation sums both contributions:

$$\frac{\partial J}{\partial \mathbf{h}_t} = \underbrace{U^\top \delta_{z,t}}_{\text{current output}} + \underbrace{W_h^\top \delta_{a,t+1}}_{\text{future recurrence}}. \tag{10}$$

This is an application of the multivariable chain rule. Backpropagation does not choose one route through a graph; every downstream route contributes to the gradient of a shared node.

4.4 Backpropagation through time

For a sequence with T supervised outputs, the total objective is often a sum or mean of token losses:

$$J = \sum_{t=1}^{T} J^{(t)}. \tag{11}$$

Backpropagation through time (BPTT) means:

1. unfold the recurrent computation over the observed sequence;
2. compute all token losses;
3. propagate gradients backward from later states to earlier states; and
4. sum every time-step contribution to the shared parameters.

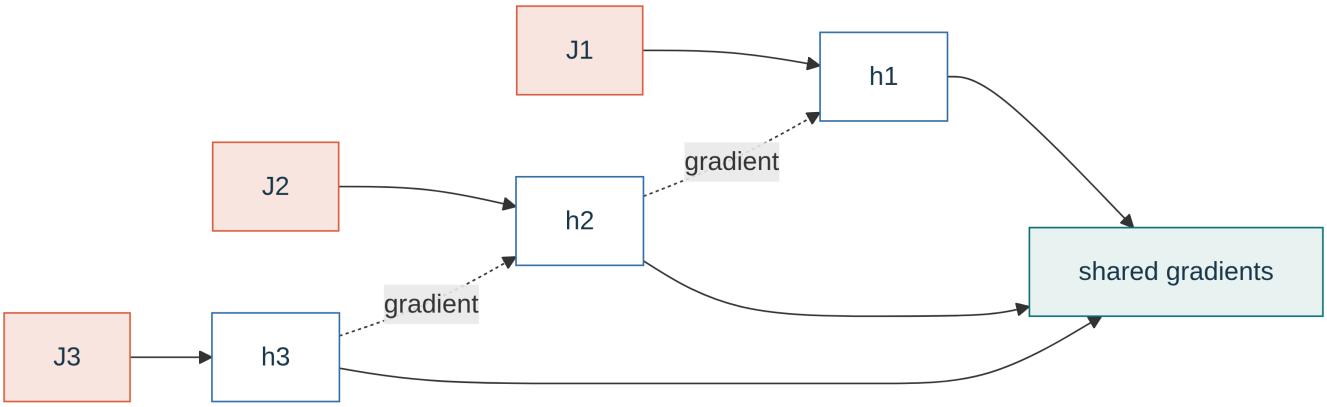


Figure 6: Each time step uses the same parameters, so its local contribution is added to one shared parameter gradient.

For example,

$$\frac{\partial J}{\partial W_h} = \sum_{t=1}^{T} \delta_{a,t} \mathbf{h}_{t-1}^{\top}. \tag{12}$$

The sum reflects weight sharing: there are many uses of W_h, but only one parameter tensor to update.

> **Implementation view**
>
> Modern automatic-differentiation systems construct this unfolded computation graph dynamically or symbolically. “Run BPTT” usually means computing the forward sequence loss and calling the framework’s backward operation—not manually writing every derivative.

5 Generating structured text with an RNN

The same recurrent mechanism can be trained at the character level. At each step, the model predicts a distribution over the next character; a sampled character is fed back as the next input. Repeating this loop generates text of arbitrary length.



Figure 7: An original visual interpretation of an absurd recipe generated in the style learned by a character-level RNN.

A recipe corpus contains highly visible regularities: titles, quantities, ingredient lists, numbered actions, punctuation, and recurring verbs. An RNN can learn these surface patterns even when its generated content is incoherent.

```
PINEAPPLE COCOA PASTA

1 cup pasta
2 tbsp cocoa powder
1 cup pineapple chunks
```

```
...

Mix until the sauce remembers the oven.
```

The sample looks recipe-like because the model has learned local and medium-range form. That appearance does not imply physical consistency, factuality, or safety. The historical recipe-generation example provides a useful seed for the coding project: inspect generated structure, then trace how temperature and sampling decisions affect output the original demonstration is available here.

6 Why gradients vanish or explode

Long-range learning requires a gradient to cross many recurrent transitions. Repeated multiplication is the central difficulty.

6.1 The repeated Jacobian product

For the simple RNN,

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} = D_t W_h,$$

where

$$D_t = \operatorname{diag}(1 - \mathbf{h}_t \odot \mathbf{h}_t)$$

contains derivatives of tanh. A gradient traveling from time T to time 0 contains

$$\frac{\partial \mathbf{h}_T}{\partial \mathbf{h}_0} = \prod_{t=1}^{T} D_t W_h. \tag{13}$$

If these local transformations repeatedly contract the signal, the gradient vanishes. If they repeatedly expand it, the gradient explodes (Bengio et al. 1994; Pascanu et al. 2013).

6.2 A scalar calculation

The matrix case changes different directions by different amounts, but a scalar chain exposes the basic behavior.

Try it first

A loss gradient has magnitude 1 at the final state and crosses four identical local derivatives.

1. What reaches the first state if every derivative is 0.6?
2. What reaches it if every derivative is 1.5?

For the contracting case,

$$1 \times 0.6^4 = 0.1296.$$

The successive magnitudes are

```
1.000 → 0.600 → 0.360 → 0.216 → 0.1296
```

For the expanding case,

$$1 \times 1.5^4 = 5.0625,$$

with

```
1.000 → 1.500 → 2.250 → 3.375 → 5.0625
```

Table 6: Both behaviors arise from repeated local transformations, not from sequence length alone.

Behavior	Training consequence
Vanishing gradient	early states receive almost no learning signal from distant losses
Exploding gradient	updates become unstable and may overflow numerically

6.3 Gradient clipping: a practical guardrail

Clipping by global norm caps an excessively large gradient while preserving its direction. Given gradient vector $mathbf{g}$ and threshold τ,

$$\tilde{\mathbf{g}} = \min\left(1, \frac{\tau}{\|\mathbf{g}\|_2}\right)\mathbf{g}. \tag{14}$$

If $\|\mathbf{g}\|_2 = 18$ and $\tau = 5$,

$$\tilde{\mathbf{g}} = \frac{5}{18}\mathbf{g}, \qquad \|\tilde{\mathbf{g}}\|_2 = 5.$$

Every component is multiplied by the same positive scalar, so the direction is unchanged.

```python
def clip_by_global_norm(gradient, threshold):
    norm = gradient.norm(p=2)
    if norm > threshold:
        gradient = (threshold / norm) * gradient
    return gradient
```

⚠ What clipping does not solve

Clipping controls explosions. It cannot restore a long-range gradient that has already faded toward zero. A more structural solution must provide a less distorted route for information and credit.

7 Gated recurrent cells

LSTM and GRU cells add learned gates that control how much old information is retained and how much new information is written. LSTM was introduced in 1997, GRU in 2014, and gated RNNs became especially prominent during roughly 2012–2018 (Hochreiter and Schmidhuber 1997; Cho et al. 2014). The Transformer appeared in 2017; replacement of recurrent models was gradual and task-dependent (Vaswani et al. 2017).

Gates use the logistic sigmoid

$$\sigma(a) = \frac{1}{1 + e^{-a}},$$

whose output lies between 0 and 1. Elementwise multiplication by a gate therefore behaves like a learned soft control: values near 0 suppress a component, while values near 1 preserve it.

7.1 LSTM: retain, write, expose

An LSTM separates the cell state $\mathbf{c}_t$ from the visible hidden state $\mathbf{h}_t$.

7.1.1 Retain or forget

$$\mathbf{f}_t = \sigma\big(W_f[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_f\big)\,.$$

$\mathbf{f}_t \odot \mathbf{c}_{t-1}$ selects how much previous memory survives.

7.1.2 Prepare a write

$$\begin{aligned}\mathbf{i}_t &= \sigma(W_i[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_i)\,,\\ \mathbf{g}_t &= \tanh\big(W_g[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_g\big)\,.\end{aligned}$$

$\mathbf{i}_t$ controls write strength; $\mathbf{g}_t$ proposes candidate content.

7.1.3 Update memory additively

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \mathbf{g}_t. \tag{15}$$

The additive path is the key structural change. When $\mathbf{f}_t$ remains near 1 and the write term remains small, information and gradient can travel through the cell state with less repeated distortion.

7.1.4 Expose part of memory

$$\begin{aligned}\mathbf{o}_t &= \sigma(W_o[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_o)\,,\\ \mathbf{h}_t &= \mathbf{o}_t \odot \tanh(\mathbf{c}_t).\end{aligned}$$

The output gate controls which parts of memory become visible to predictions and the next recurrent step.

7.2 GRU: reset, propose, interpolate

A GRU combines memory and visible state into one vector. It uses two main gates.

7.2.1 Reset the old state before proposing a candidate

$$\mathbf{r}_t = \sigma(W_r[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_r)\,,$$

$$\tilde{\mathbf{h}}_t = \tanh(W_h[\mathbf{r}_t \odot \mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_h)\,.$$

7.2.2 Interpolate between old and proposed state

$$\mathbf{z}_t = \sigma(W_z[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_z)\,,$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t. \tag{16}$$

The update is a learned elementwise mixture. A small component of $\mathbf{z}_t$ keeps the corresponding old-state component; a large value replaces it with the candidate.

Table 7: Both architectures create learned paths that can preserve information; neither guarantees that every long dependency will be learned.

Property	LSTM	GRU
Persistent representation	cell state $\mathbf{c}_t$ plus hidden state $\mathbf{h}_t$	hidden state $\mathbf{h}_t$ only
Main controls	forget, input, output	reset, update
State update	additive retain + write	interpolation between old and candidate

Property	LSTM	GRU
Typical advantage	explicit memory highway	simpler gated computation

Coding perspective

Libraries expose LSTM and GRU modules because the internal equations are reused at every time step. In implementation, inspect tensor shapes, initial states, returned sequences, and padding masks rather than reimplementing every gate unless the exercise explicitly asks for it.

8 Neural machine translation

Machine translation (MT) generates a sentence in a target language that preserves the meaning of a source sentence. For example,

```
SOURCE · French   Le colis arrivera demain.
TARGET · English  The package will arrive tomorrow.
```

The task is not word substitution. The output must express the source meaning while obeying the grammar and word order of the target language.

Google's 2016 Neural Machine Translation system was a major production-scale turning point: an end-to-end neural system replaced the previous phrase-based pipeline for deployed translation (Wu et al. 2016). This was not the invention of neural MT, but it made the transition visible at global scale. Translation is now embedded in messaging, web access, search, meetings, captions, travel, and support systems.

8.1 Encoder–decoder recurrence

An encoder reads the source sequence and updates a state:

$$\mathbf{h}_t^{\text{enc}} = f_{\text{enc}}(\mathbf{x}_t, \mathbf{h}_{t-1}^{\text{enc}}).$$

After the final source token, the last encoder state becomes a context vector:

$$\mathbf{c} = \mathbf{h}_{T_x}^{\text{enc}}. \tag{17}$$

The decoder starts from that context and generates the target sequence:

$$\mathbf{s}_t = f_{\text{dec}}(\mathbf{e}(y_{t-1}), \mathbf{s}_{t-1}, \mathbf{c}),$$

$$P(y_t \mid y_{<t}, \mathbf{x}) = \text{softmax}(U\mathbf{s}_t + \mathbf{b}).$$

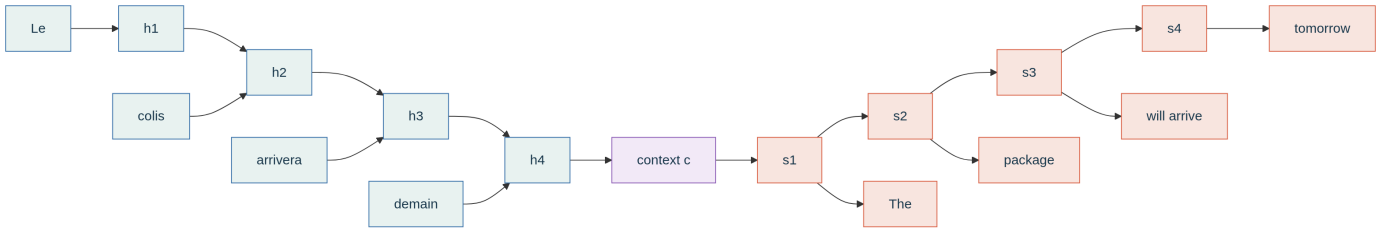


Figure 8: The final encoder state initializes a decoder that generates a new sequence. Encoder and decoder may have separate parameters.

The source and target may have different lengths. The architecture defines two roles—reading and writing—not a requirement that one parameter matrix perform both roles (Sutskever et al. 2014; Cho et al. 2014).

9 Conditional language models

Translation is one instance of a broader encoder–decoder pattern:

$$\text{encode input } x \quad \longrightarrow \quad \text{generate output } y.$$

The interpretation of x and y defines the task.

Table 8: The same conditional-generation abstraction supports different input and output spaces.

Task	Input x	Generated output y	Representative work
Summarization	document	shorter meaning-preserving summary	Rush et al. (2015)
Dialogue	conversation history	next response	Vinyals and Le (2015)
Parsing as generation	sentence	linearized syntax tree	Vinyals et al. (2015)
Code generation	natural-language specification	program	Yin and Neubig (2017)

For parsing, the qualification matters: a parser becomes a conditional language model when its structure is represented as a sequence of output tokens. Not every parsing architecture is autoregressive.

These systems are called **conditional language models** because they factorize an output sequence autoregressively while conditioning every prediction on x:

$$P(\mathbf{y} \mid \mathbf{x}) = \prod_{t=1}^{T_y} P(y_t \mid y_{<t}, \mathbf{x}). \tag{18}$$

It is a **language model** because it predicts y token by token. It is **conditional** because the input $\mathbf{x}$ remains part of every next-token distribution.

9.1 Training seq2seq end to end

During supervised training, each target token provides a decoder-side cross-entropy loss:

$$J^{(t)} = -\log P(y_t \mid y_{<t}, \mathbf{x}).$$

The sequence loss is

$$J = \sum_{t=1}^{T_y} J^{(t)}. \tag{19}$$

No token-level translation loss is attached directly to encoder states. Nevertheless, backpropagation follows the full computation graph:

$$J \longrightarrow \text{decoder states} \longrightarrow \mathbf{c} \longrightarrow \text{encoder states and parameters.}$$

Thus, **loss location** and **update reach** are different questions:

- losses are computed where target tokens are predicted;
- decoder parameters receive direct downstream credit;
- the context path receives gradient from the decoder; and
- encoder parameters are updated because they produced the context on which every decoder prediction depends.

> **Teacher forcing**
>
> During basic training, the decoder commonly receives the observed previous target token y_{t-1} rather than its own sampled prediction. This makes all target-side losses computable in parallel across the unrolled training graph, although recurrent states themselves remain sequential. At inference time, the true next tokens are unavailable, so generated tokens must be fed back.

10 The fixed-context bottleneck

The early encoder–decoder design forces the entire source through one fixed-size vector:

$$\mathbf{x}_{1:T_x} \longrightarrow \boxed{\mathbf{c}} \longrightarrow \mathbf{y}_{1:T_y}.$$

For a short sentence, this may work well. As the source becomes longer or information-rich, $\mathbf{c}$ must preserve every detail that any later decoder step might require. The decoder cannot return to a particular source state; it can consult only the compressed summary.

The limitation

The decoder may need one specific source detail, but the architecture requires all details to compete for representation inside the same fixed-size context vector.

This bottleneck motivates the next chapter. Attention was introduced in neural machine translation to let generation depend on source information more selectively (Bahdanau et al. 2015). We stop before its mechanism.

10.1 Chapter synthesis

The chapter followed a sequence of architectural repairs:

Table 9: Each repair solves one problem while exposing the next.

Limitation	Repair	Remaining question
exact n-gram counts do not share evidence	embeddings and shared neural parameters	how can context length vary?
fixed neural window grows with position count	recurrent state and weight sharing	how does distant credit survive?
repeated recurrent Jacobians vanish or explode	clipping and gated cells	how should one sequence condition another?
one fixed context summarizes the source	encoder–decoder conditional LM	how can the decoder retrieve specific source information?

An RNN replaces a fixed input window with a learned state. BPTT trains that state across time, but repeated transformations make long-range credit fragile. LSTM and GRU cells create gated, partly additive routes for information. Encoder–decoder models then turn recurrence into a general conditional-generation framework. Their fixed-context bottleneck leaves one question for the next chapter.

Carry this question forward

Can a decoder retrieve the relevant source information at each output step instead of demanding that one vector remember everything?

10.2 References

Bahdanau, Dzmitry, Kyunghyun Cho, and Yoshua Bengio. 2015. “Neural Machine Translation by Jointly Learning to Align and Translate.” *International Conference on Learning Representations*. https://arxiv.org/abs/1409.0473.

Bengio, Yoshua, Réjean Ducharme, Pascal Vincent, and Christian Jauvin. 2003. “A Neural Probabilistic Language Model.” *Journal of Machine Learning Research* 3: 1137–55. https://jmlr.org/papers/v3/bengio03a.html.

Bengio, Yoshua, Patrice Simard, and Paolo Frasconi. 1994. “Learning Long-Term Dependencies with Gradient Descent Is Difficult.” *IEEE Transactions on Neural Networks* 5 (2): 157–66. https://doi.org/10.1109/72.279181.

Cho, Kyunghyun, Bart van Merriënboer, Caglar Gulcehre, et al. 2014. “Learning Phrase Representations Using RNN Encoder–Decoder for Statistical Machine Translation.” *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, 1724–34. https://doi.org/10.3115/v1/D14-1179.

Hochreiter, Sepp, and Jürgen Schmidhuber. 1997. “Long Short-Term Memory.” *Neural Computation* 9 (8): 1735–80. https://doi.org/10.1162/neco.1997.9.8.1735.

Jurafsky, Daniel, and James H. Martin. 2026. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*. 3rd ed. https://web.stanford.edu/~jurafsky/slp3/.

Pascanu, Razvan, Tomas Mikolov, and Yoshua Bengio. 2013. “On the Difficulty of Training Recurrent Neural Networks.” *Proceedings of the 30th International Conference on Machine Learning*, Proceedings of machine learning research, vol. 28: 1310–18. https://proceedings.mlr.press/v28/pascanu13.html.

Rush, Alexander M., Sumit Chopra, and Jason Weston. 2015. “A Neural Attention Model for Abstractive Sentence Summarization.” *Proceedings of EMNLP*, 379–89. https://aclanthology.org/D15-1044/.

Sutskever, Ilya, Oriol Vinyals, and Quoc V. Le. 2014. “Sequence to Sequence Learning with Neural Networks.” *Advances in Neural Information Processing Systems* 27. https://papers.nips.cc/paper/5346-sequence-to-sequence-learning-with-neural-networks.

Vaswani, Ashish, Noam Shazeer, Niki Parmar, et al. 2017. “Attention Is All You Need.” *Advances in Neural Information Processing Systems* 30. https://arxiv.org/abs/1706.03762.

Vinyals, Oriol, Łukasz Kaiser, Terry Koo, Slav Petrov, Ilya Sutskever, and Geoffrey Hinton. 2015. “Grammar as a Foreign Language.” *Advances in Neural Information Processing Systems* 28. https://papers.nips.cc/paper/5635-grammar-as-a-foreign-language.

Vinyals, Oriol, and Quoc V. Le. 2015. “A Neural Conversational Model.” *arXiv Preprint arXiv:1506.05869*. https://arxiv.org/abs/1506.05869.

Wu, Yonghui, Mike Schuster, Zhifeng Chen, et al. 2016. “Google’s Neural Machine Translation System: Bridging the Gap Between Human and Machine Translation.” *arXiv Preprint arXiv:1609.08144*. https://arxiv.org/abs/1609.08144.

Yin, Pengcheng, and Graham Neubig. 2017. “A Syntactic Neural Model for General-Purpose Code Generation.” *Proceedings of ACL*, 440–50. https://aclanthology.org/P17-1041/.